ESP32-C3 Breadboard Adapter

Release 5.8

Alexander Bobkov

Nov 17, 2024

CONTENTS:

1 About **3**

2 Schematic **5**
2.1 GPIOs 7

3 Examples **9**
3.1 MicroPython Blinky Example 9

4 Learning Activities **11**

5 Bill of Materials **13**

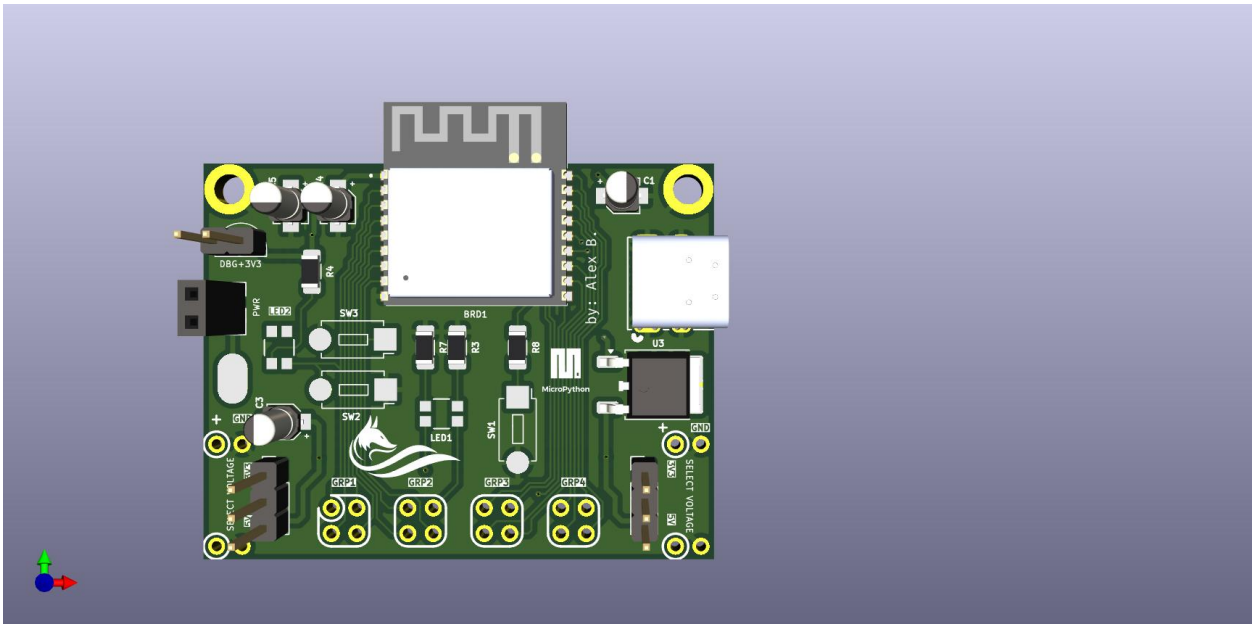
6 Index **15**

Add your content using `reStructuredText` syntax. See the reStructuredText documentation for details.

CHAPTER

ONE

ABOUT



Features:

- compact size requires very little space on a breadboard;
- connects to the power rails on a breadboard;
- can be powered by USB-C port or external power supply;
- 3.3V and 5V power supplies;
- GPIOs arranged into 4 groups for easier identification;
- on-board ESP32-C3 Module with USB-C port allows to flash a firmware as you work with breadboard;
- debugging points;
- power pads makes it easy to connect alligators supplying voltage to the board;
- on-board system LEDs: power, and module reset mode;
- on-board push switch for Reset and Flash modes;
- on-board push switch and LED for your needs;

ESP32-C3 Module:

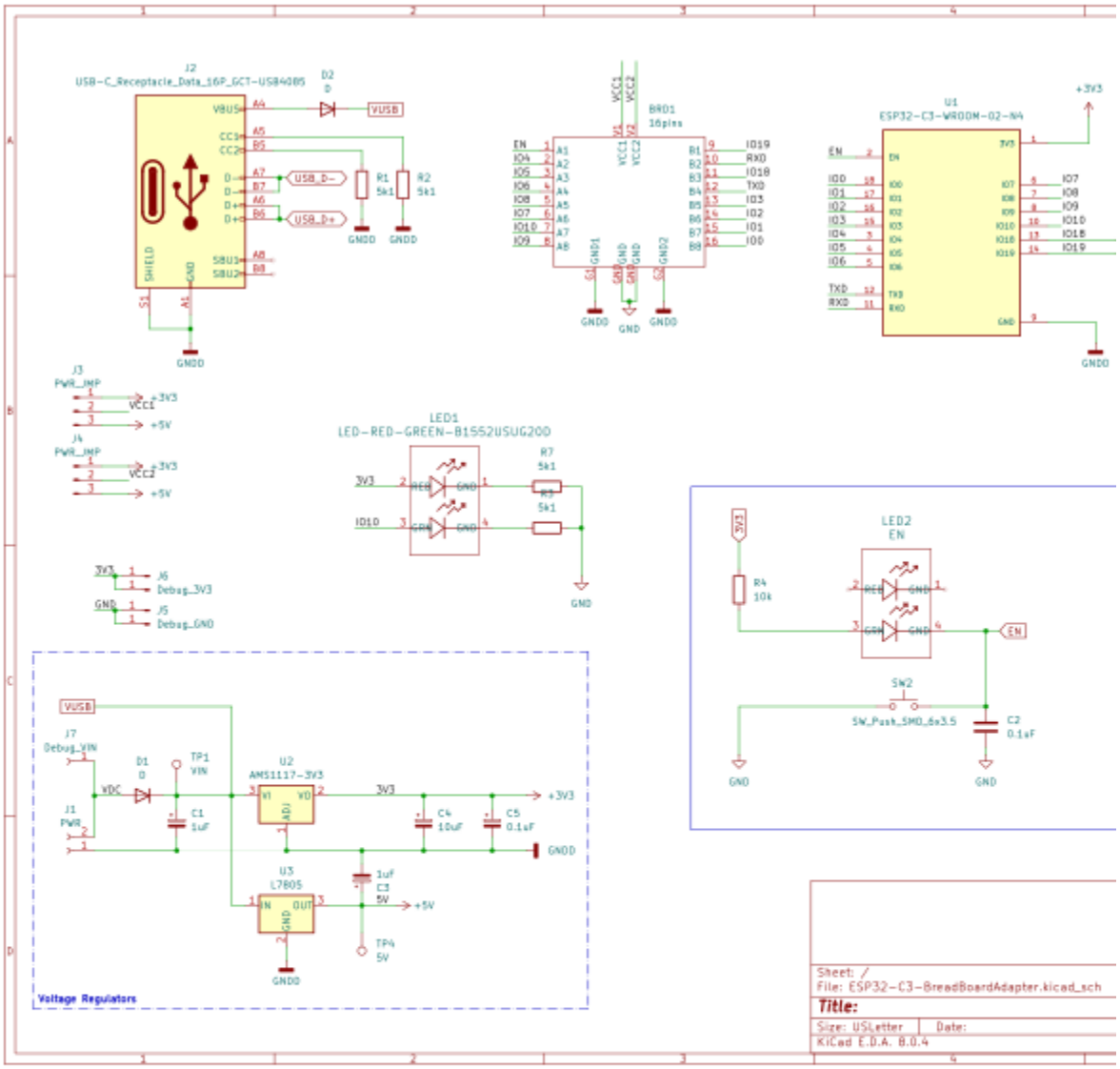
- single-core

- 32-bit RISC-V MCU @ 160 MHz
- 400 KB of internal RAM
- RSA-3072-based secure boot and the AES-128/256-XTS flash encryption
- low power-mode support
- Rich connectivity for IoT applications: Wi-Fi and Bluetooth 5 (LE) with long-range support
- Bluetooth LE SIG Mesh and Wi-FI Mesh support
- 22 GPIOs

CHAPTER

TWO

SCHEMATIC



2.1 GPIOs

Table 1: GPIOs

Pin	GPIO	Description
13	GPIO18	USB_D-
14	GPIO19	USB_D+
10	GPIO10	User-defined LED
TXD	TXD	UART Transmit data
RXD	RXD	UART Receive data

CHAPTER

THREE

EXAMPLES

3.1 MicroPython Blinky Example

```python
import machine
from machine import Pin
import time, math

ONBOARD_LED = 10    # GPIO10, PIN 7
ONBOARD_BTN = 3     # GPIO3, 13

# Configure on-board LED and push button
# Stated GPIOs correspond to the wiring schematic
onboard_button = Pin(ONBOARD_BTN, Pin.IN, Pin.PULL_UP)

led = machine.PWM(ONBOARD_LED, freq=1000)

def pulse(l, t):
    for i in range(20):
        l.duty(int(math.sin(i/10 * math.pi) * 500 + 500))
        time.sleep_ms(t)
    l.duty(0)

# Interrupt function to turn LED ON when on-board button is pressed
def button_interrupt(pin):
    pulse(led, 70)

def main():

    # Assign interrupt to the on-board push button
    onboard_button.irq(trigger=Pin.IRQ_FALLING, handler=button_interrupt)

if __name__ == '__main__':
    main()
```

CHAPTER

FOUR

LEARNING ACTIVITIES

CHAPTER

FIVE

BILL OF MATERIALS

The table below lists components required for this project.

Table 1: Bill of Materials

Item	Part #	Qty	Description
Row 1, Column 1			Row 1, Column 2
Row 2, Column 1	Row 2, Column 2	Row 2, Column 3	Row 2, Column 4

CHAPTER

SIX

INDEX

- genindex
- search